Разработка информационного приложения

Города и Страны



Актуальность

Цель – быстрый поиск качественных данных о городах и странах

Потенциальные пользователи:

1. путешественники
2. сотрудники международных компаний
3. люди, интересующие географией









Используемые технологии

1

Изученные ранее:

- PyQT6 (QPixmap, QImage, QMessageBox, QInputDialog, QTableWidgetItem, QLabel, QComboBox, QLCDNumber, QLineEdit, QPushButton, QButtonGroup, QStatusBar и т.д.);
- datetime;
- sqlite3;
- pyinstaller:
- csv.

2

Изученные самостоятельно:

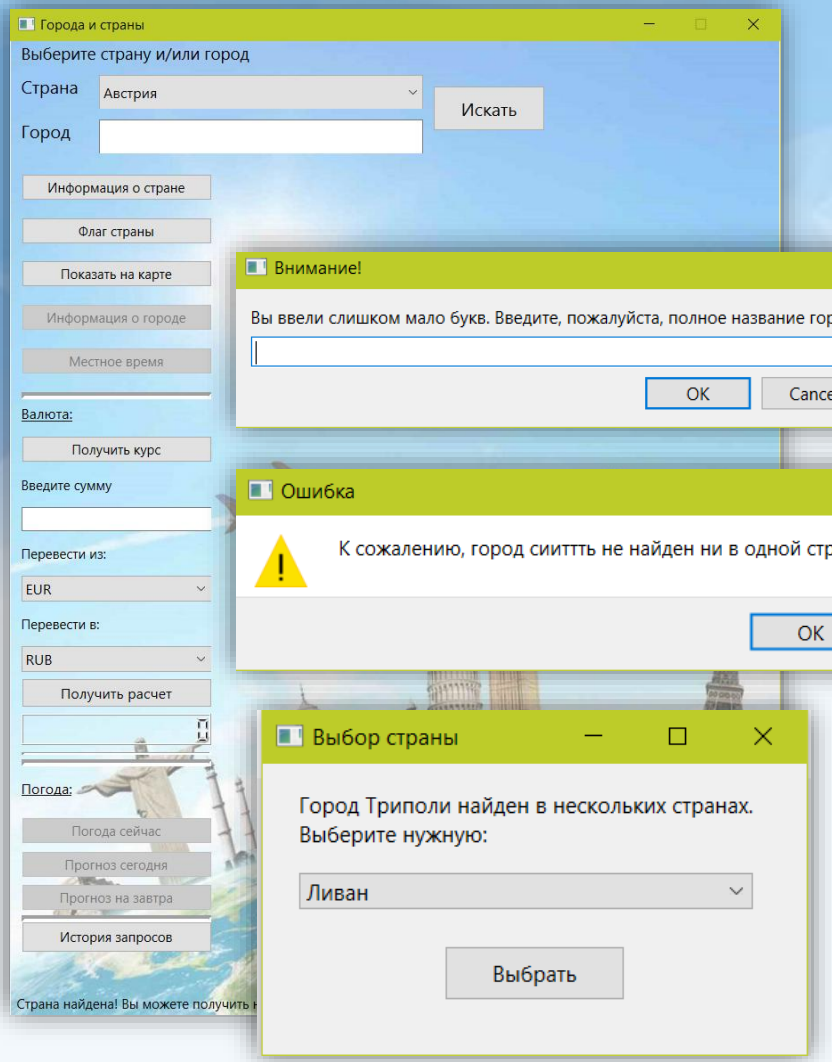
- PyQT6 (QTextBrowser, QTimer, QDesktopServices, Qwidget и т.д.);
- pyqtgraph;
- requests;
- pytz;
- countryinfo;
- translate;
- работа с данными в формате json.

Структура приложения

1. Класс Main
2. Классы UiMainWindow и SecondWindow
3. Класс PlaceSearch
4. Классы Country и City
5. Классы Currency и Weather
6. Прочие файлы:

- history_dt.sqlite
- country.csv
- languages.txt
- Пояснительная записка.docx
- requirements.txt
- cities_and_countries.exe

Интерфейс приложения. Выбор места поиска




```python
def get_search_res(self):
    # возвращаем первоначальный вид приложения
    self.clear_widgets()
    self.disable_widgets()
    self.chosen_city = None
    self.chosen_country = None
    self.statusBar().showMessage("")
    # считываем введенные данные
    self.country_input = self.countreComboBox.currentText()
    self.city_input = self.cityEntry.text()
    # проверяем ввели ли страну
    got_country = True if self.country_input != "Не выбрана" else False
    # проверяем правильность ввода города
    got_city = True if len(self.city_input) > 2 else False
    # если введено меньше двух букв, просим заполнить снова
    if 0 < len(self.city_input) < 3:
        while not got_city:
            inp, ok_pressed = QInputDialog.getText(self, "Внимание!",
                                                   "Вы ввели слишком мало букв.
                                                   "название города.")
            if ok_pressed:
                if len(inp) > 2:
                    self.cityEntry.setText(inp)
                    self.city_input = inp
                    got_city = True
    # если выбрана только страна
    if got_country and not got_city:
        self.search_by_country()
    # если выбран город и страна
    if got_country and got_city:
        self.search_by_country_and_city()
    # если ищем только по названию города
    if not got_country and got_city:
        self.search_by_city()
```

Информация о стране

Города и страны

Выберите страну и/или город

Страна: Австрия

Искать

Город:



Информация о стране

Флаг страны

Показать на карте

Информация о городе

Местное время

Австрия находится в регионе Европа,
столица - город Вена,
население - 8.53 миллионов человек,
валюта - Евро,
движение правостороннее,
официальный язык - немецкий язык
больше информации доступно на сайте Википедии

1 € (EUR, Евро) равен 110.20 рублям и 1.05 долларам США.

Валюта:

Получить курс

Введите сумму

1000

Перевести из:

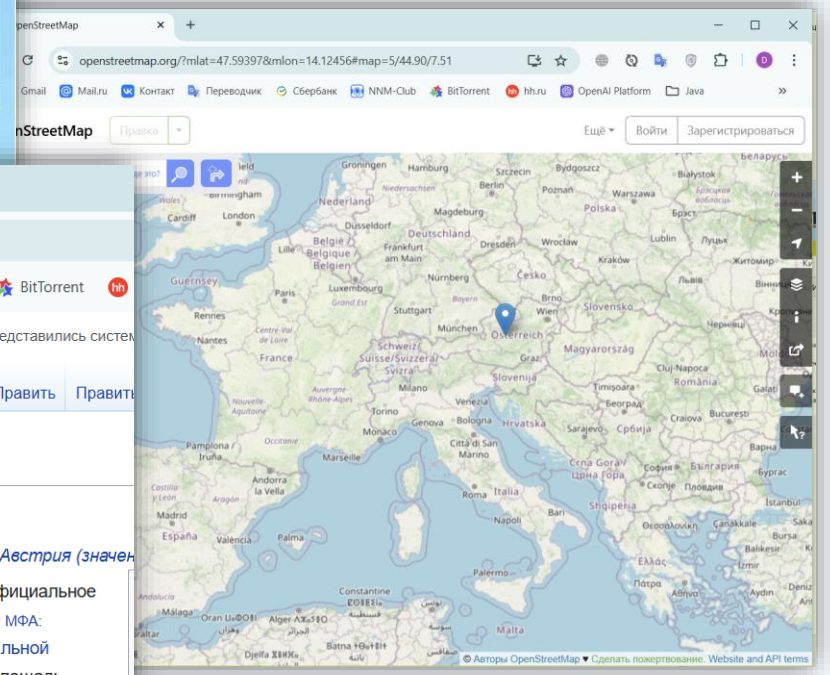
EUR

Перевести в:

RUB

Получить расчет

110202.44



OpenStreetMap

Австрия — Википедия

ru.wikipedia.org/wiki/Австрия

Gmail Mail.ru Контакт Переводчик Сбербанк NNM-Club BitTorrent

ВикипедиЯ
Свободная энциклопедия

Статья Обсуждение Читать Править

Австрия

Материал из Википедии — свободной энциклопедии

У этого термина существуют и другие значения, см. Австрия (значен

А́встрия[7] (нем. *Österreich* МФА: ['øːstɐˌʁaɪç] ⓘ), официальное название — **Австри́йская Респу́блика** (*Republik Österreich* МФА: [repuˈbliːk ˈʔøːstɐʁaɪç] ⓘ) — государство в Центральной Европе. Население составляет 8,858 млн человек (2019), площадь — 83 879 км²[8]. Занимает 98-е место в мире по численности населения и 113-е по территории.

На севере граничит с Чехией (362 км), на северо-востоке — со Словакией (91 км), на востоке — с Венгрией (366 км), на юге — со

Заглавная страница
Содержание
Избранные статьи
Случайная статья
Текущие события
Пожертвовать

Участие
Сообщить об ошибке
Как править статьи
Сообщество
Форум

Информация о городе

Города и страны

Выберите страну и/или город

Страна: Италия

Искать

Город: Рим

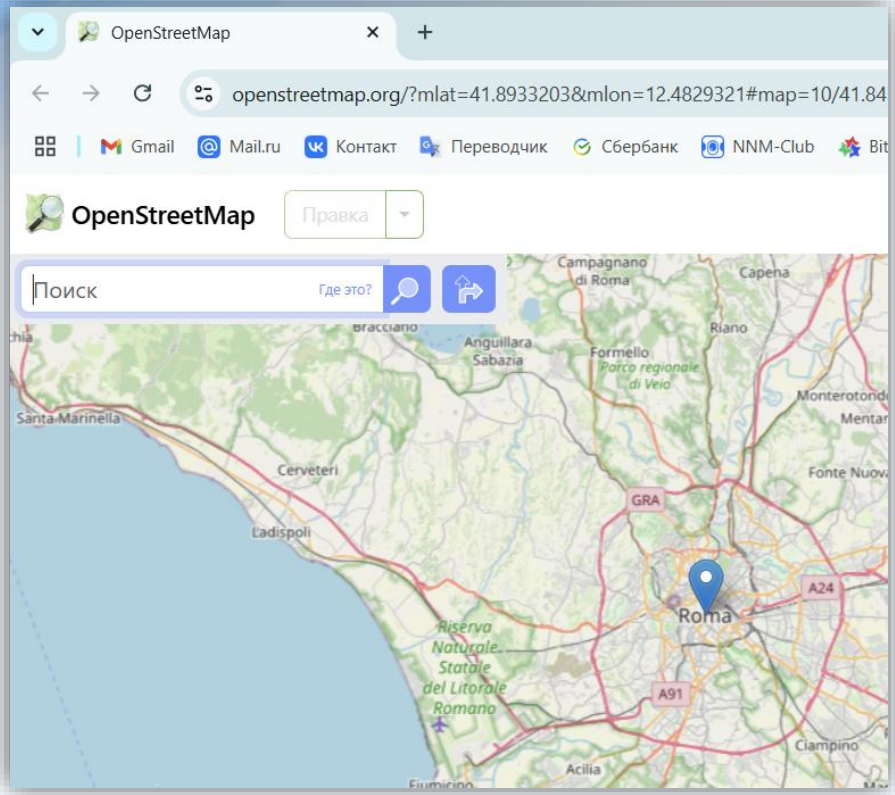
Информация о стране

Флаг страны

Показать на карте

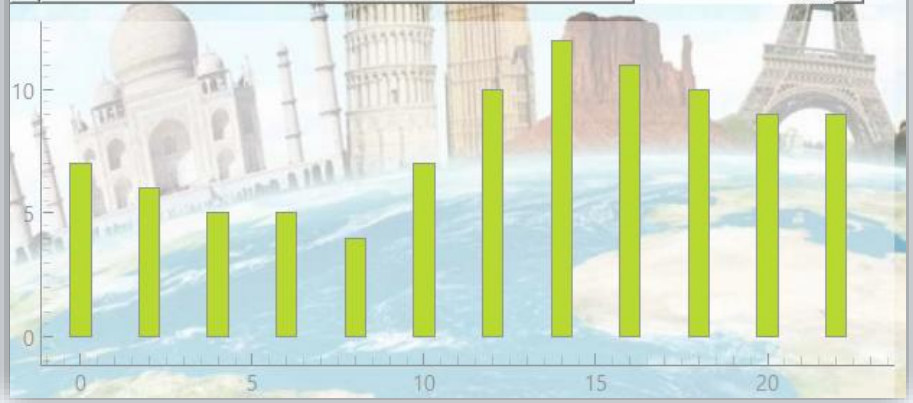
17:34:34

Часовой пояс в городе Рим - Европа/Рим (CET), +0100.
Разница во времени с текущим местоположением -2 ч



Прогноз погоды на завтра, 2024-12-06, отражена в таблице.
Динамика температуры воздуха по часам также отражена на диаграмме.

	Темпетарура°C	Осадки мм	Облачность %	Ветер м/с
0ч	7	0	15	6
2ч	6	0	13	6
4ч	5	0	0	5
6ч	5	0	100	3
8ч	4	0	86	3
10ч	7	0	68	3
12ч	10	0	100	3
14ч	12	0	100	3



Получение информации. API.

Используемые API:

1. https://api.opencagedata.com– координаты,
2. основная информация, часовой пояс и т.д.;
3. https://open.er-api.com - курсы валют;
4. https://api.open-meteo.com – прогноз погоды
5. https://flagpedia.net/ - флаги.

Free for non-commercial

Open-Meteo APIs are free for non-commercial use. We do not restrict access, but ask for fair use and attribution.

Kindly ensure that your daily requests remain below 10,000. Please note that we retain the right to block applications and IP addresses that misuse our service. For additional information, please refer to the terms and conditions.

Commercial Use License

Commercial clients enjoy the advantages of dedicated API servers and personalized support. If you have any inquiries or require assistance, please feel free to reach out to us at info@open-meteo.com.

During the evaluation, prototyping, and development stages, we encourage you to utilize the free tier. However, for your production system, we will provide you with an API key to ensure that your application reliably receives weather data.

Библиотека requests, получение словаря:

```
class Currency:
    @staticmethod
    def get_currency_rates(currency):
        currency_code = currency["code"]
        currency_name = currency["name"]
        currency_symbol = currency["symbol"]
        try:
            response = requests.get(f'https://open.er-api.com/v6/latest/{currency_code}'
            response.raise_for_status()
            data = response.json()
```

Специализированная библиотека:

```
from opencage.geocoder import OpenCageGeocode


class PlaceSearch:
    GEOKEY = OpenCageGeocode("53532ba5f03b48b0b9d3c633e44d97d4")

    # ищем страну
    @staticmethod
    def find_country(country):
        data = PlaceSearch.GEOKEY.geocode(country, language="ru")
        # если нашли страну, собираем информацию и возвращаем ее
        if data:
            lat = data[0]["geometry"]["lat"]
            lng = data[0]["geometry"]["lng"]
```

Библиотека requests, получение картинки:

```
def get_flag(self):
    flag_URL = f"https://flagcdn.com/160x120/{self.code}.png"
    flag = requests.get(flag_URL).content
```

Получение информации. Библиотеки countryinfo, translate, pytz.

1. Получение и перевод информации о стране:

```
from translate import Translator
from countryinfo import CountryInfo
```

```
def get_country_info(self):
    to_eng = Translator(from_lang="ru", to_lang="en")
    to_rus = Translator(from_lang="en", to_lang="ru")
    name_eng = to_eng.translate(self.name)
    country = CountryInfo(name_eng)
    capital = to_rus.translate(country.capital())
    languages = country.languages()
    population = round(country.population() / 1000000, 2)
```

2. Получение и объекта часового пояса и местного времени:

```
self.timezone_pytz = pytz.timezone(tz_name)
local = datetime.now(self.timezone_pytz)
```

Вывод информации

1. местное время

```
self.timer = QTimer(self)
```

```
def show_local_time(self):
    self.widget.setVisible(False)
    self.tableWidget.setVisible(False)
    self.textBrowser.setCurrentCharFormat(self.text_format)
    tz_info = self.chosen_city.get_time_info()
    self.textBrowser.setText(tz_info)
    self.timer.timeout.connect(self.show_clock)
    self.timer.start(1000)

def show_clock(self):
    clock = self.chosen_city.get_current_time()
    self.clock_label.setText(clock.strftime('%X'))
```

2. гиперссылка

```
link = f'<a href={wiki}>Википедии</a><br>'
```

```
self.textBrowser.anchorClicked.connect(QDesktopServices.openUrl)
self.text_format = self.textBrowser.currentCharFormat()
```

```
def show_info(self):
    self.widget.setVisible(False)
    self.tableWidget.setVisible(False)
    self.textBrowser.clear()
    self.textBrowser.setCurrentCharFormat(self.text_format)
    if self.sender() == self.countryInfo:
        text, link = self.chosen_country.get_country_info()
    else:
        text, link = self.chosen_city.get_city_info()
    self.textBrowser.insertPlainText(text)
    self.textBrowser.insertHtml(link)
```

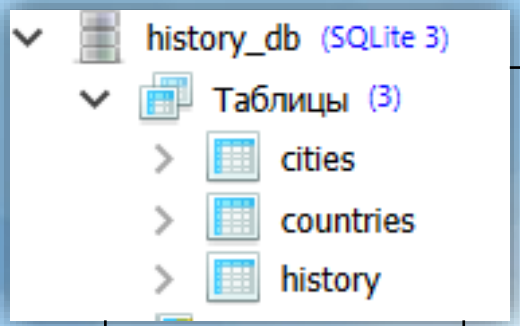
3. график

```
def show_forecast(self):
    if self.bargraph:
        self.widget.removeItem(self.bargraph)
self.bargraph = pyqtgraph.BarGraphItem(x=int_hours, height=temp, width=0.6, brush='b')
self.widget.addItem(self.bargraph)
```

4. карта

```
def show_map(self):
    if self.chosen_city == "":
        map_url = f"https://www.openstreetmap.org/?mlat={self.chosen_country.lat}&mlon={self.chosen_country.lng}"
    else:
        map_url = f"https://www.openstreetmap.org/?mlat={self.chosen_city.lat}&mlon={self.chosen_city.lng}"
    webbrowser.open(map_url)
```

Сохранение истории запросов



history_db | Имя таблицы: history

	Имя	Тип данных	Первичный ключ	Внешний ключ
1	id	INTEGER	🔑	
2	created	TEXT		
3	country	INTEGER		🔗
4	city	INTEGER		🔗

history_db | Имя таблицы: countries | W

	Имя	Тип данных	Первичный ключ	Внешний ключ	Уникальность
1	id	INTEGER	🔑		
2	name	TEXT			🔗
3	region	TEXT			
4	currency	TEXT			

history_db | Имя таблицы: cities | WITHOUT ROWID | S

	Имя	Тип данных	Первичный ключ	Внешний ключ	Уникальность	Проверка	Не NULL
1	id	INTEGER	🔑				
2	name	TEXT			🔗		🔒
3	country	INTEGER		🔗	🔗		🔒

	Тип	Имя	Подробности
1	UNIQUE		(name, country)

```python
def save_search(self):
    # заполняем таблицу стран, если такой страны там еще нет
    country_name = self.chosen_country.name
    region = self.chosen_country.region
    currency = self.chosen_country.currency["ru_name"]
    self.cur.execute( _sql: '''INSERT OR IGNORE INTO countries(name, region, currency)
                               VALUES(?, ?, ?)''',
                      _parameters: (country_name, region, currency))
    self.con.commit()
    # заполняем таблицу города, если он выбран и отсутствует в базе данных
    city_name = self.city_input.title()
    if city_name != "":
        self.cur.execute(
            _sql: '''INSERT OR IGNORE INTO cities(name, country)
                     VALUES(?, (SELECT id FROM countries WHERE name = ?))''',
            _parameters: (city_name, country_name))
        self.con.commit()
    # заполняем таблицу с историей
    created = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    self.cur.execute( _sql: '''
    WITH    country_id AS (SELECT id FROM countries WHERE name = ?),
            city_id AS (SELECT id
                        FROM cities
                        WHERE name = ?
                        AND country = (SELECT * FROM country_id))
    INSERT
    INTO history (created, country, city)
    VALUES (?,
            (SELECT * FROM country_id),
            (SELECT * FROM city_id)
                                    )''',
                      _parameters: (country_name, city_name, created))
    self.con.commit()
```

Вывод истории запросов

```
def show_history(self):
    self.textBrowser.setVisible(False)
    self.tableWidget.setVisible(True)
    self.widget.setVisible(False)
    result = self.cur.execute("""
                SELECT history.created,
                    countries.name,
                    COALESCE(cities.name, "Не выбрано"),
                    countries.region,
                    countries.currency
                FROM history
                LEFT JOIN countries ON history.country = countries.id
                LEFT JOIN cities ON history.city = cities.id
                ORDER BY history.id DESC
                """).fetchmany(15)
```

Время и дата	Страна	Город	Регион	Валюта
2024-12-05 18:21:44	Индонезия	Не выбрано	Азия	Индонезийская Рупия
2024-12-05 18:15:59	Италия	Рим	Европа	Евро
2024-12-05 16:34:20	Италия	Рим	Европа	Евро
2024-12-05 16:27:51	Австрия	Не выбрано	Европа	Евро
2024-12-05 16:27:25	Италия	Не выбрано	Европа	Евро
2024-12-05 16:18:32	Италия	Рим	Европа	Евро
2024-12-05 16:17:03	Италия	Рим	Европа	Евро
2024-12-05 16:16:27	Италия	Рим	Европа	Евро
2024-12-05 16:11:03	Италия	Рим	Европа	Евро

Возможности для доработки

1. Добавление функции поиска отелей и авиабилетов по выбранной локации;
2. Добавление информации о лучших ресторанах и обзора популярных достопримечательностей;
3. Расчет расстояния от текущего местоположения до выбранной локации.
4. Демонстрация истории курса национальной валюты к популярным валютам на графике;
5. Доработка интерфейса, загрузка информации из базы данных при повторном поиске.